Raspberry Pi Imager での Image 作成手順

（OTG 接続対応を含む）

Raspberry Pi OS (Trixie)　Pi5 / Pi4 / Zero2W（ZeroW）共通版

Raspberry Pi OS（Trixie）で、新しい Image を作成するための手順書です。OTG 接続は、PC と Raspberry Pi 4 だけでなく、Raspberry Pi Zero2W や Raspberry Pi 5 を USB ケーブルだけで接続できる大変便利な方法です。

※ ラズパイ 5 では電源の流れる電流が大きいので、データ転送対応の特別な USB-C ケーブルの準備が必要です。

※ Zero W は 32bit 環境のため、64bit 版の OS は使用できません。必ず 32bit 版を選択してください。

項目	内容
対象ハードウェア	Raspberry Pi5 / Pi4 / Zero2W（ZeroW）
OS	Raspberry Pi OS (Debian Trixie) Pi5 / Pi4 / Zero2W : 64bit 版　Zero W : 32bit 版
書き込みツール	Raspberry Pi Imager 2.0.6 以降
PC クライアント	Windows 11
ターミナル	Tera Term
VNC クライアント	RealVNC Viewer

※ ZeroW + Trixie では VNC は動作しますが、動作が重いためあまりおすすめしません。

作成日：2026 年 3 月 20 日　最終更新日：2026 年 3 月 31 日

Created by @M164 using Claude

目次

番号	内容
(1)	PC（Windows11）用 USB Ethernet ドライバのインストール
(2)	Raspberry Pi Imager のインストール
(3)	microSD カードのフォーマット（再使用する場合）
(4)	Raspberry Pi Imager で Trixie イメージを書き込む
(5)	有線 LAN または Wi-Fi 接続の準備
(6)	ラズパイに microSD カードを挿入して USB ケーブルで PC と接続する
(7)	OTG 接続を有効化する
(8)	PC（Windows11）のネットワーク共有の設定
(9)	raspi-config で VNC を有効化する
(10)	VNC から接続する
(11)	Localization の設定（書き込み時の設定が反映されないので）
(12)	時間の設定（NTP サーバーの設定）
(13)	OS のアップデート
(14)	日本語入力メソッド（fcitx5-mozc）の設定
(15)	alias の設定
(16)	OTG 接続の状態確認
(17)	OTG 接続の ON/OFF 方法
(18)	VNC 解像度の設定（1600×900）
(19)	OS アップデート後の解像度リセット対策
(20)	GPIO21 プルアップ設定と電源オフスクリプトの自動起動（IoT 学習 HAT 使用時）
(21)	各機種の起動ビット数と config.txt の設定について
任意	カスタマイズ（fastfetch / gparted / WiringPi）
	トラブルシューティング
	便利な確認コマンド

(1) PC（Windows11）用の USB Ethernet ドライバのインストール

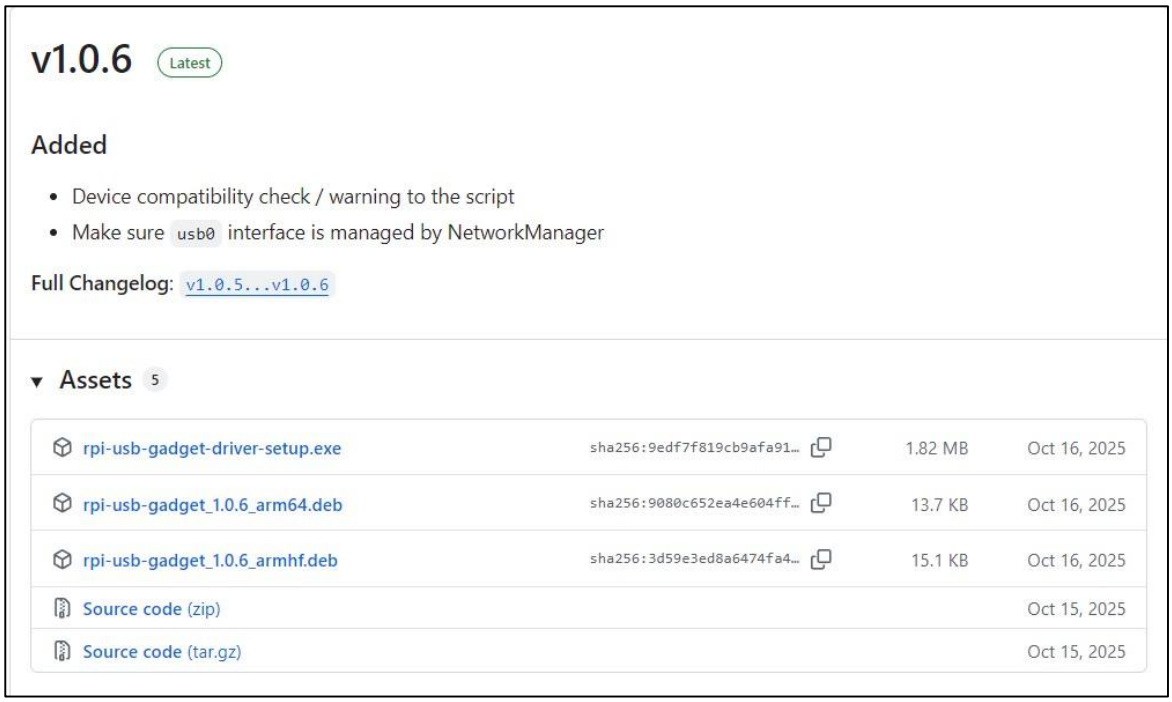
ラズパイ公式の Windows 11 用「Raspberry Pi USB RNDIS Driver」をインストールします。

以下の URL からダウンロードして実行します：

```
https://github.com/raspberrypi/rpi-usb-gadget/releases
```

「rpi-usb-gadget-driver-setup.exe」をダウンロードして実行します。

ダウンロードページ：



v1.0.6 Latest

Added

- Device compatibility check / warning to the script
- Make sure `usb0` interface is managed by NetworkManager

Full Changelog: `v1.0.5...v1.0.6`

▾ Assets 5

rpi-usb-gadget-driver-setup.exe	sha256:9edf7f819cb9afa91…	1.82 MB	Oct 16, 2025
rpi-usb-gadget_1.0.6_arm64.deb	sha256:9080c652ea4e604ff…	13.7 KB	Oct 16, 2025
rpi-usb-gadget_1.0.6_armhf.deb	sha256:3d59e3ed8a6474fa4…	15.1 KB	Oct 16, 2025
Source code (zip)			Oct 15, 2025
Source code (tar.gz)			Oct 15, 2025

(2) PC（Windows11）に Raspberry Pi Imager 2.0.6 をインストール

以下の URL からダウンロードしてインストールします：

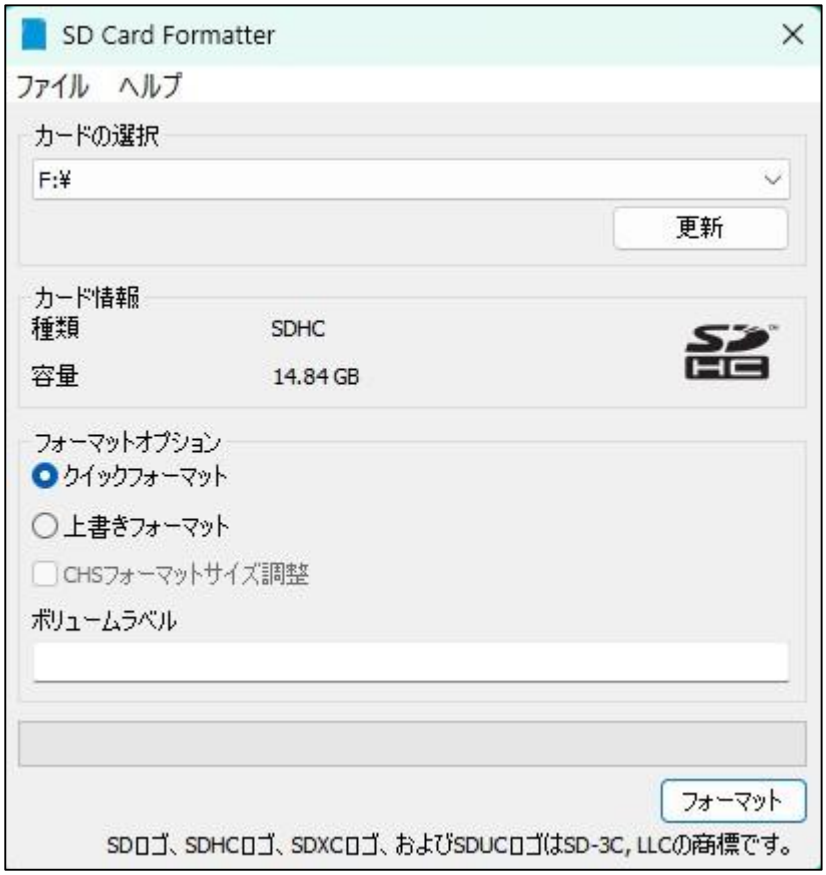
```
https://www.raspberrypi.com/software/
```

(3) microSD カードのフォーマット（再使用する場合）

使用済みの microSD カードを再利用する場合は、SD Card Formatter でフォーマットします。

```
https://www.sdcard.org/ja/downloads-2/formatter-2/
```

SD Card Formatter 画面：



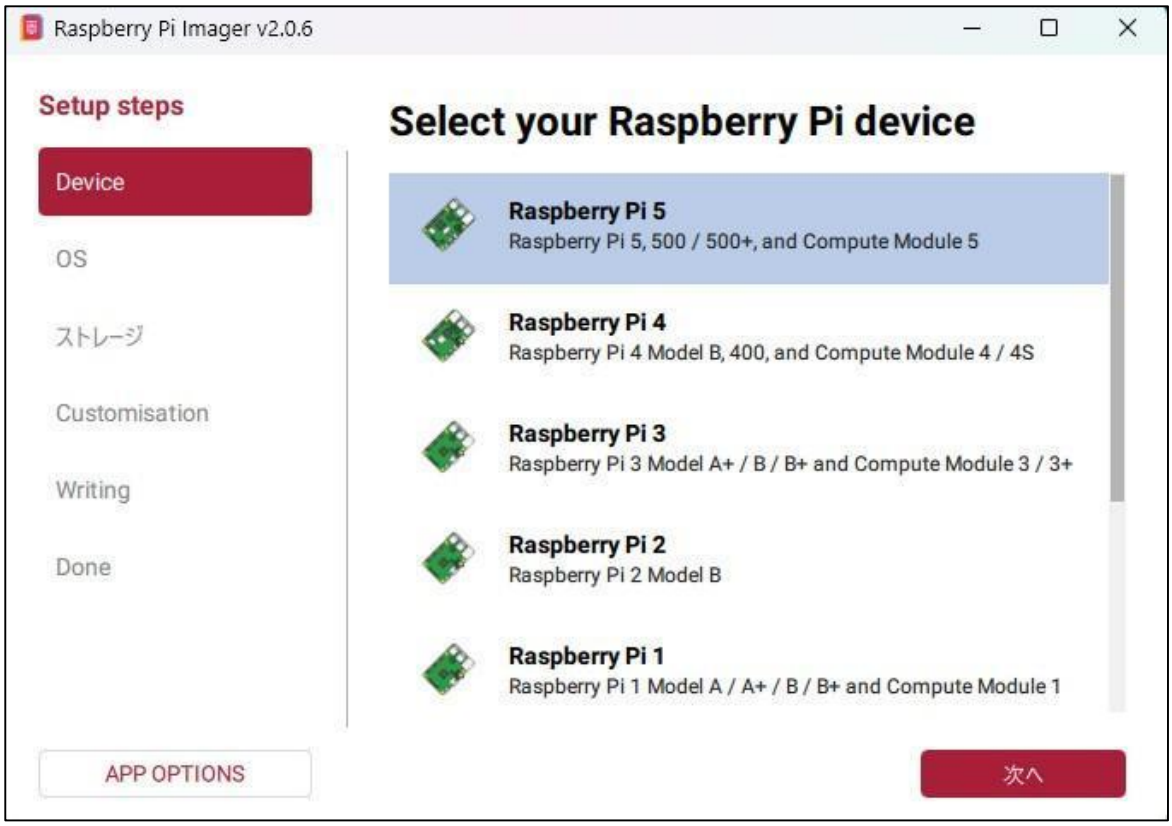
(4) Raspberry Pi Imager を使って microSD カードに Trixie イメージを書き込む

【デバイス・OS・ストレージの選択】

以下の順で選択します：

・Device で使用する Raspberry Pi を選択（Pi5 / Pi4 / Zero2W）

デバイス選択画面：

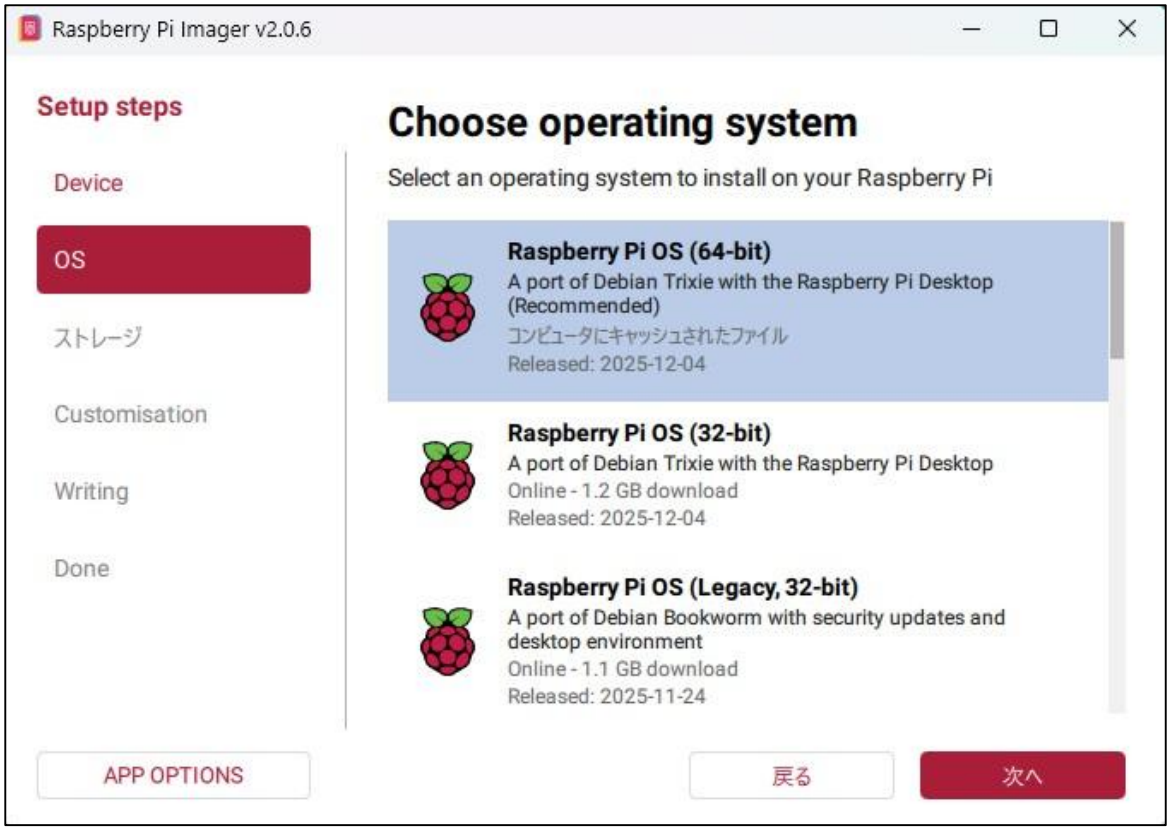


・OS を選択

・Pi5 / Pi4 / Zero2W の場合：一番上の「Raspberry Pi OS（64bit）」を選択してください。

・ZeroW の場合：「Raspberry Pi OS（32bit）」を選択してください。

OS 選択画面：

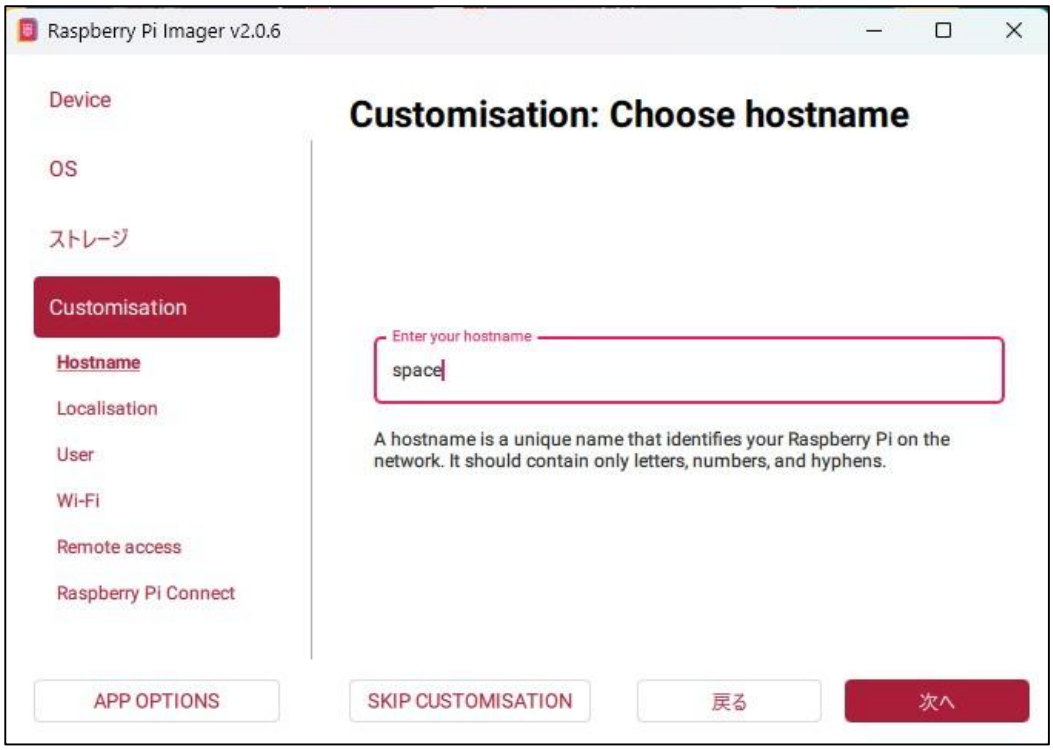


・microSD カードを選択

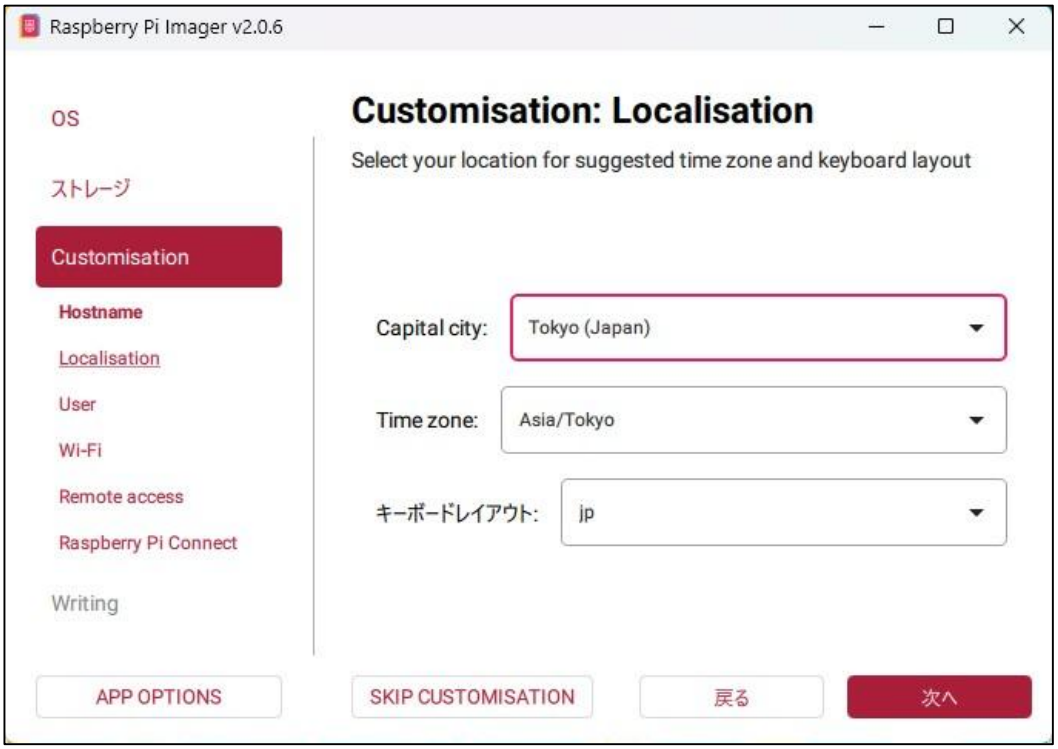
【カスタマイズ設定】

Customisation：Choose hostname

・ホスト名を入力（例：space）



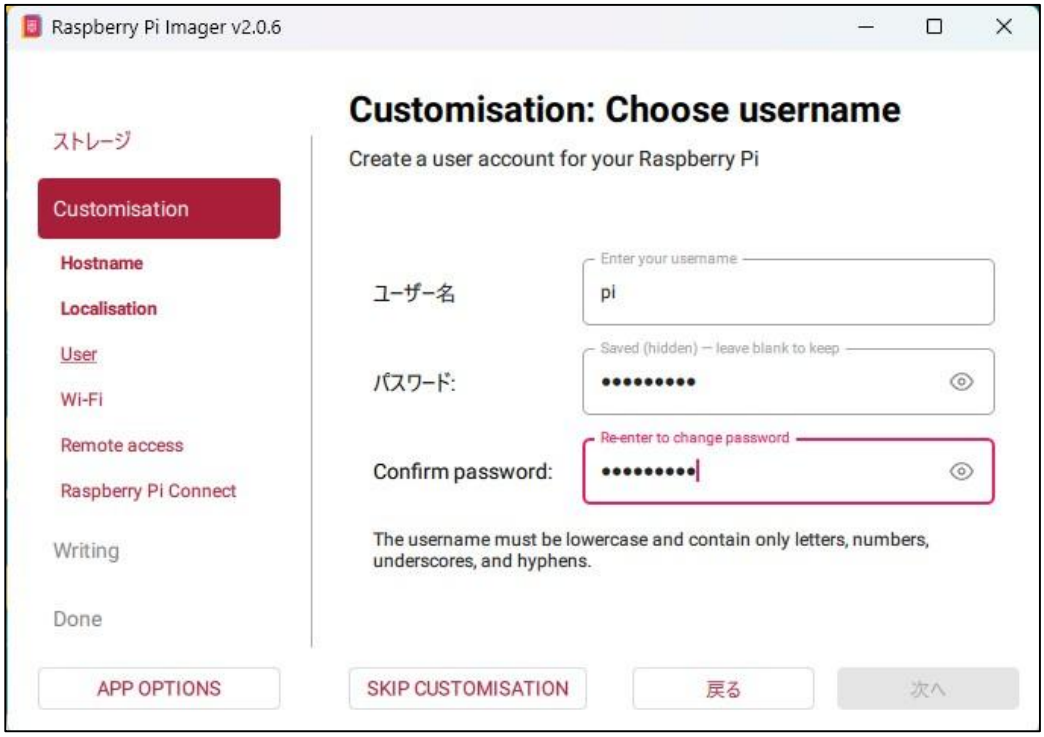
Customisation：Localisation



・Capital city に Tokyo（Japan）を入力（その他の情報は自動で入力されます）

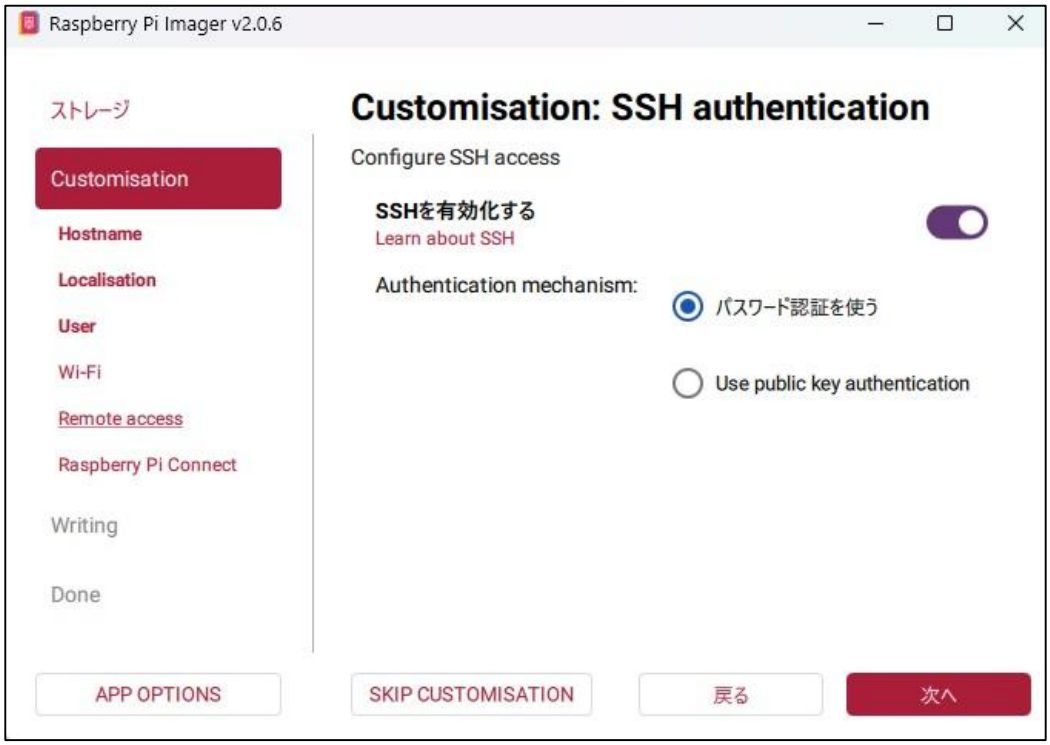
Customisation : Choose username

・ユーザ名「pi」とパスワードを入力



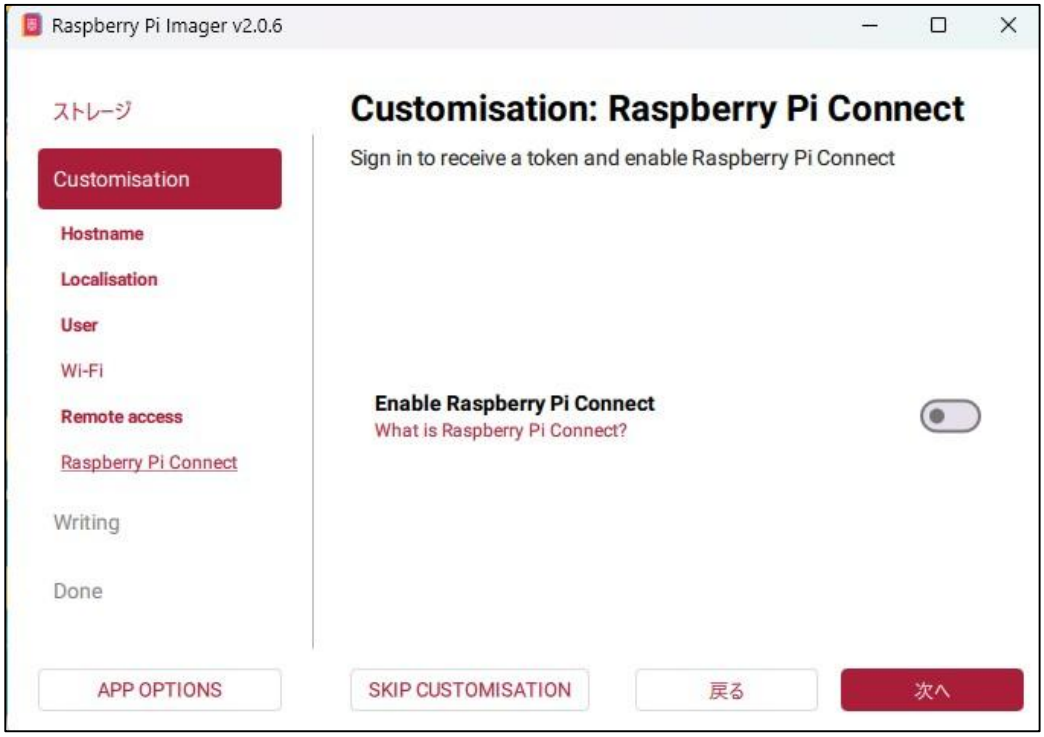
Customisation : SSH authentication

・SSH を有効化する（パスワード認証を行うにチェック）



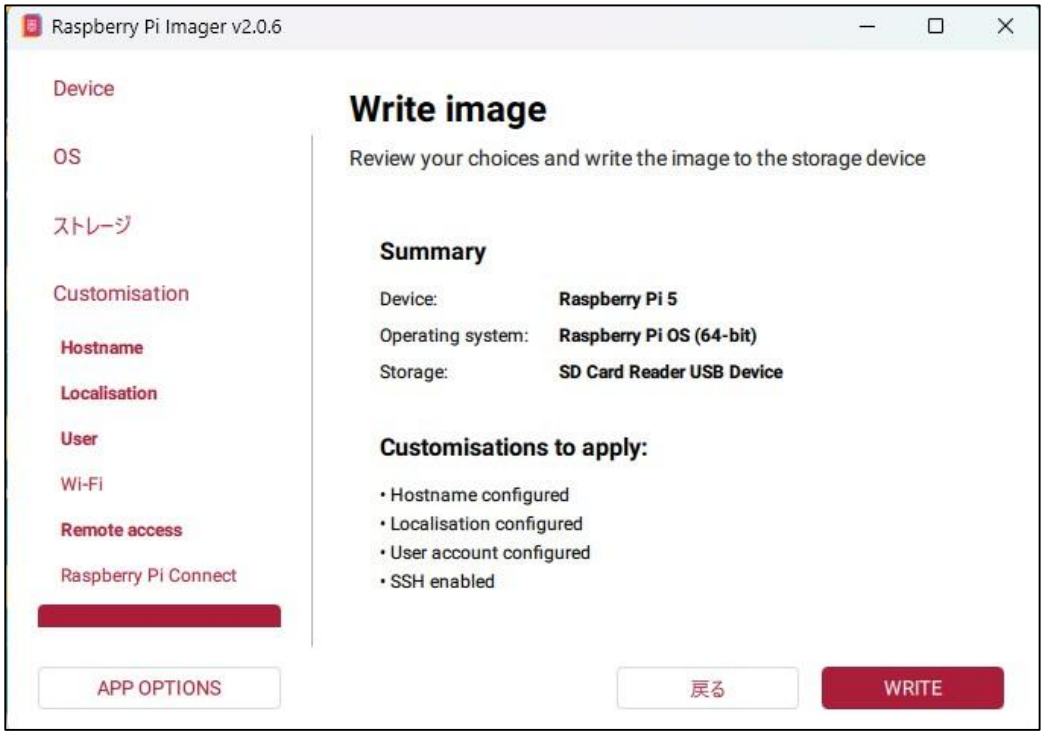
Customisation：Raspberry Pi Connect

・「Enable Raspberry Pi Connect」は OFF のまま



Write image（書き込み確認）

・「WRITE」を選択



書き込み実行

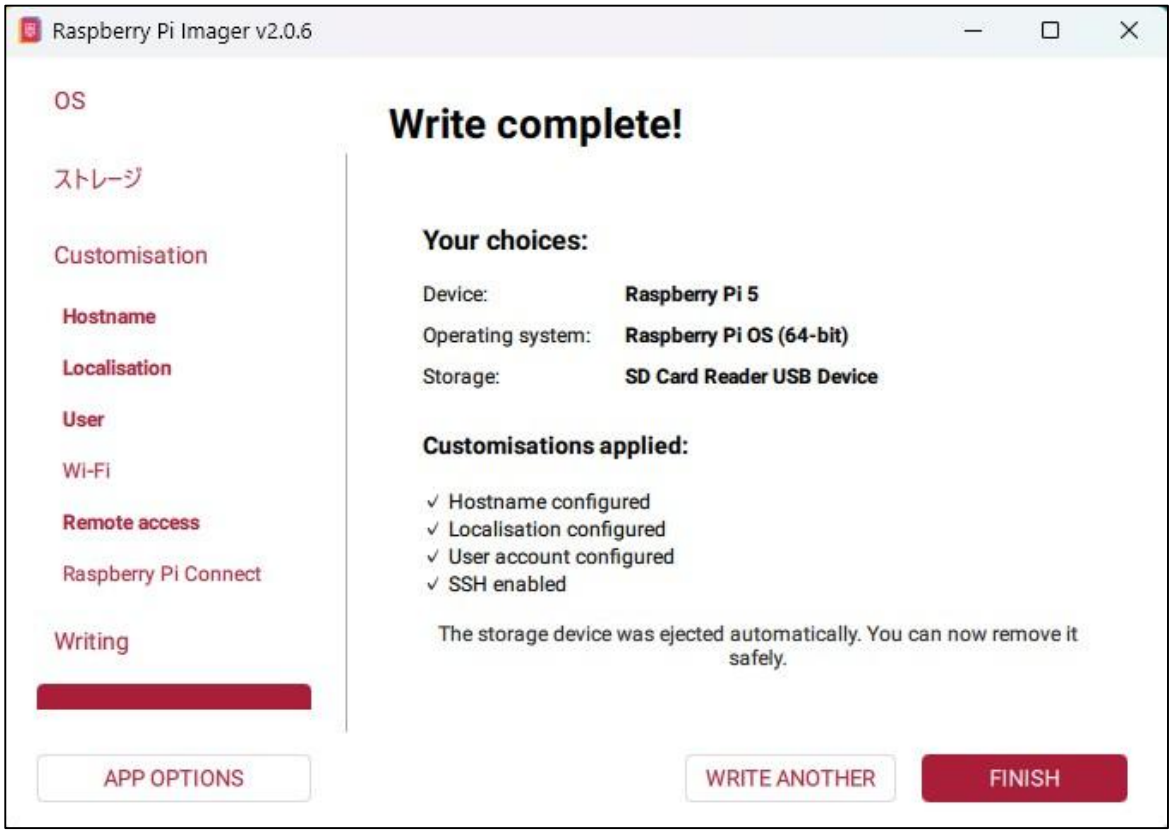
・「I UNDERSTAND, ERASE & WRITE」を選択します。

・約 5 分で書き込みが終了し、約 3 分で確認が終了します。

書き込み終了

・「FINISH」を押して終了（自動的にアンマウントされる）

・microSD カードを取り出してください。



(5) 有線 LAN または Wi-Fi 接続の準備

初回セットアップ時は OTG 接続がまだ有効になっていないため、最初の SSH ログインには有線 LAN または Wi-Fi によるネットワーク接続が必要です。

有線 LAN で接続する場合

・ラズパイの LAN ポートに LAN ケーブルを接続し、ルーターまたはスイッチに繋げます。

・ラズパイが起動すると、DHCP により自動的に IP アドレスが割り当てられます。

・ホスト名（例：space.local）または IP アドレスで SSH 接続できます。

Wi-Fi で接続する場合

・Raspberry Pi Imager のカスタマイズ設定で Wi-Fi の SSID とパスワードを設定しておきます。

・ラズパイが起動すると自動的に Wi-Fi に接続します。

・ホスト名（例：space.local）または IP アドレスで SSH 接続できます。

> ※ OTG 接続の有効化（手順(7)）が完了した後は、USB ケーブルのみで SSH / VNC 接続が可能になります。

(6) ラズパイに microSD カードを挿入して USB ケーブルで PC と接続して起動する

【USB ケーブル接続前に以下を準備する】

・スタートメニューを右クリックして「デバイスマネージャー」を選択して開いておく

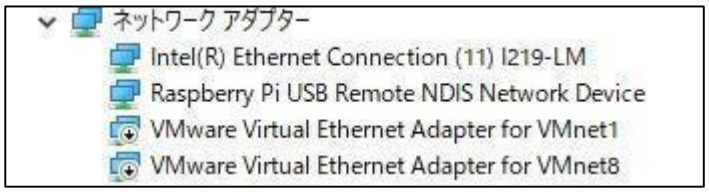
・「Win」キー＋「R」を同時に押して「ncpa.cpl」と入力し、ネットワーク接続画面を開いておく

【接続後の確認】

ラズパイを USB ケーブルに接続してから約 5 分待ち、デバイスマネージャーとネットワーク接続画面を確認します。

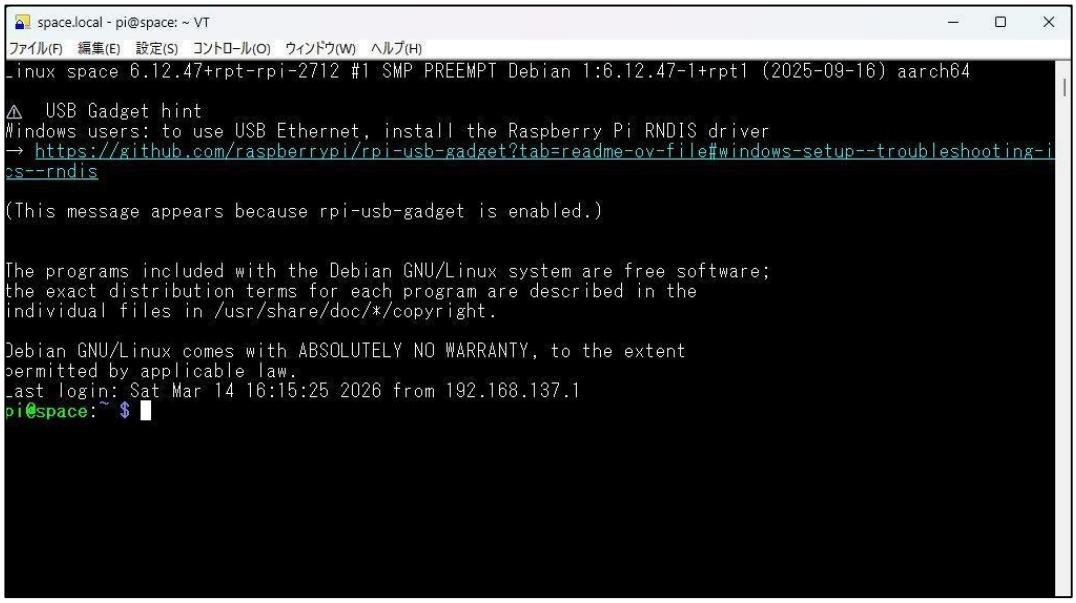
・デバイスマネージャーのネットワークアダプターに「Raspberry Pi USB Remote NDIS Network Device」が表示されることを確認

デバイスマネージャー確認：



・ネットワーク接続画面に「Raspberry Pi Remote NDIS Network Device」が表示されることを確認

ネットワーク接続確認：（Tera Term で ssh pi@space.local と入力する）



※ 約 5 分しても表示が変わらない場合は、USB ケーブルをはずして再度接続してください。

(7) OTG 接続を有効化する

有線 LAN または Wi-Fi で SSH ログインし、以下の手順で OTG 接続を有効化します。

> ※ この手順は Raspberry Pi Imager で書き込んだ直後の初回セットアップ時に必要です。

Step 1：config.txt のバックアップを取る

実行前に必ず config.txt のバックアップを取ります：

```
sudo cp /boot/firmware/config.txt /boot/firmware/config.txt.bak
```

Step 2：OTG 接続を有効化する

```
sudo rpi-usb-gadget on
```

Step 3：config.txt の内容を確認する

実行後は、以下のコマンドで config.txt の内容を確認してください：

```
cat /boot/firmware/config.txt
```

以下の行が含まれているか確認してください：

```
dtoverlay=vc4-kms-v3d
```

> ※ この行が消えていた場合、VNC が起動できなくなります（灰色画面）。その場合は config.txt を修正してください。詳細は手順（17）を参照してください。

Step 4：再起動する

```
sudo reboot
```

再起動後、OTG 接続が有効になります。

> ※ sudo rpi-usb-gadget on を実行すると config.txt が大幅に書き換えられる場合があります。そのため Step 1 のバックアップを必ず実施してください。
>
> ※「インターネット接続の共有」の設定でラズパイからインターネットに接続できますが、つながらない場合は、一度、共有のチェックマークをはずして OK を押して解除した後、再度共有の設定を行ってみてください。

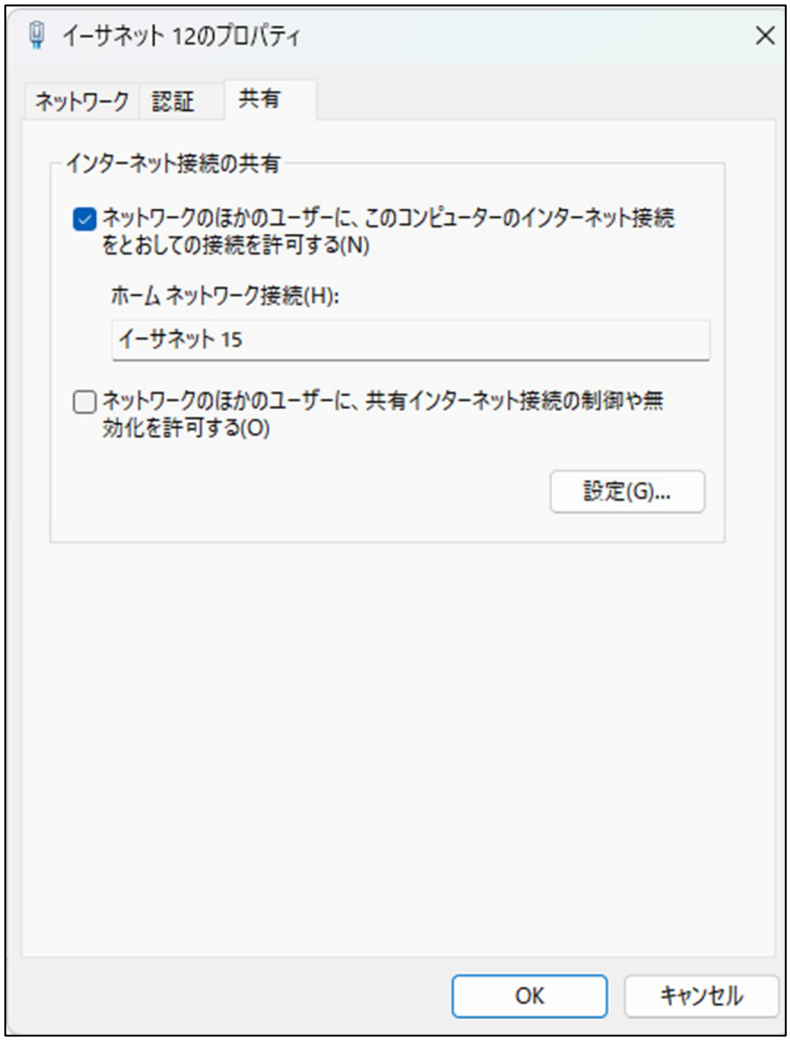
(8) PC（Windows11）のネットワーク共有の設定

メインでインターネット接続しているネットワークアダプターの「共有」の設定を行います。

共有するネットワークアダプター名は「Raspberry Pi Remote NDIS Network Device」の上にある名前を使用します。

上の図の場合は、外部に繋がっている[イーサーネット 12]のプロパティで共有タブを開きます。

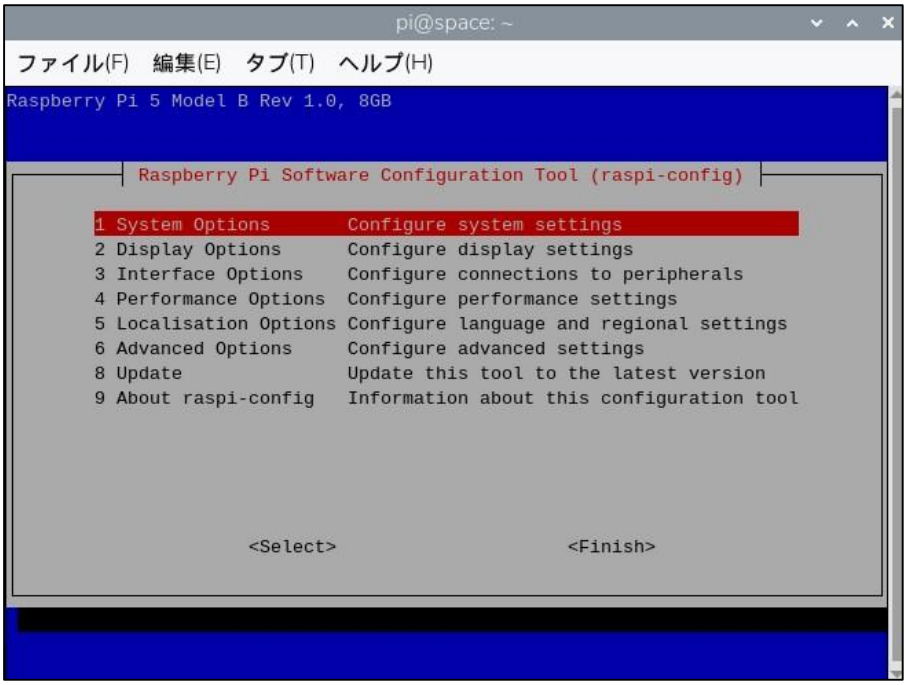
Raspberry Pi USB Remote の[イーサーネット 15]と共有してください。



(9) raspi-config で VNC を有効化する

OTG 接続で USB ケーブルを使って SSH ログインし、raspi-config を起動します：

```
sudo raspi-config
```



「3 Interface Options」→「I3 VNC」を選択して VNC を Enable にし、Finish → 再起動します。

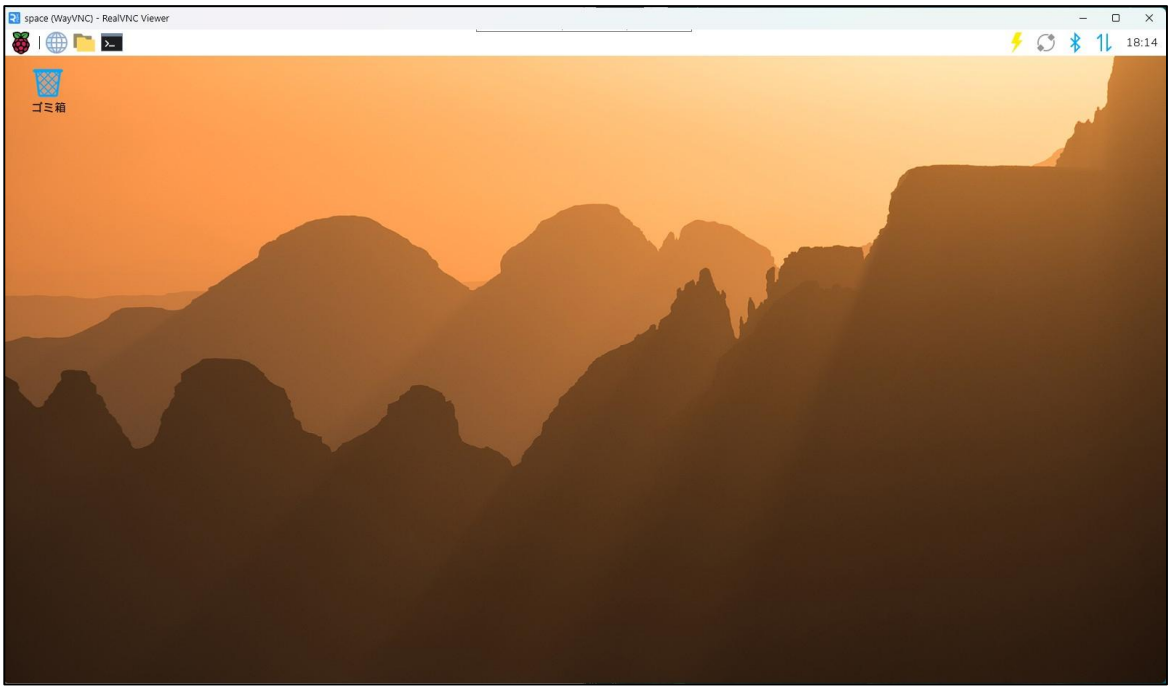
raspi-config VNC 設定画面：

(10) VNC から接続する

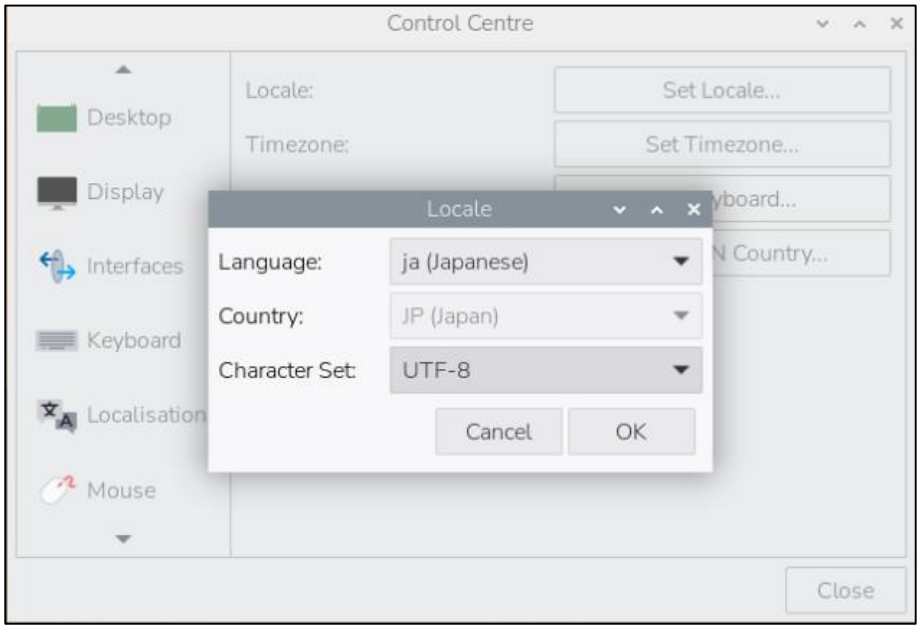
RealVNC Viewer でホスト名、ユーザ名（pi）、パスワードを入力してログインします。

```
space.local
```

以下の画面が出れば成功です：



(11) Localization の設定（書き込み時の設定が反映されないので）



Control Center で Language を「jp（Japanese）」、Character Set を「UTF-8」に変更します。
（CLI では ja_JP.UTF-8 に相当します）。

(12) 時間の設定（NTP サーバーの設定）

アップデート・アップグレードを行うには、NTP サーバーの設定をして正確な時刻を取得するために設定が必要です。

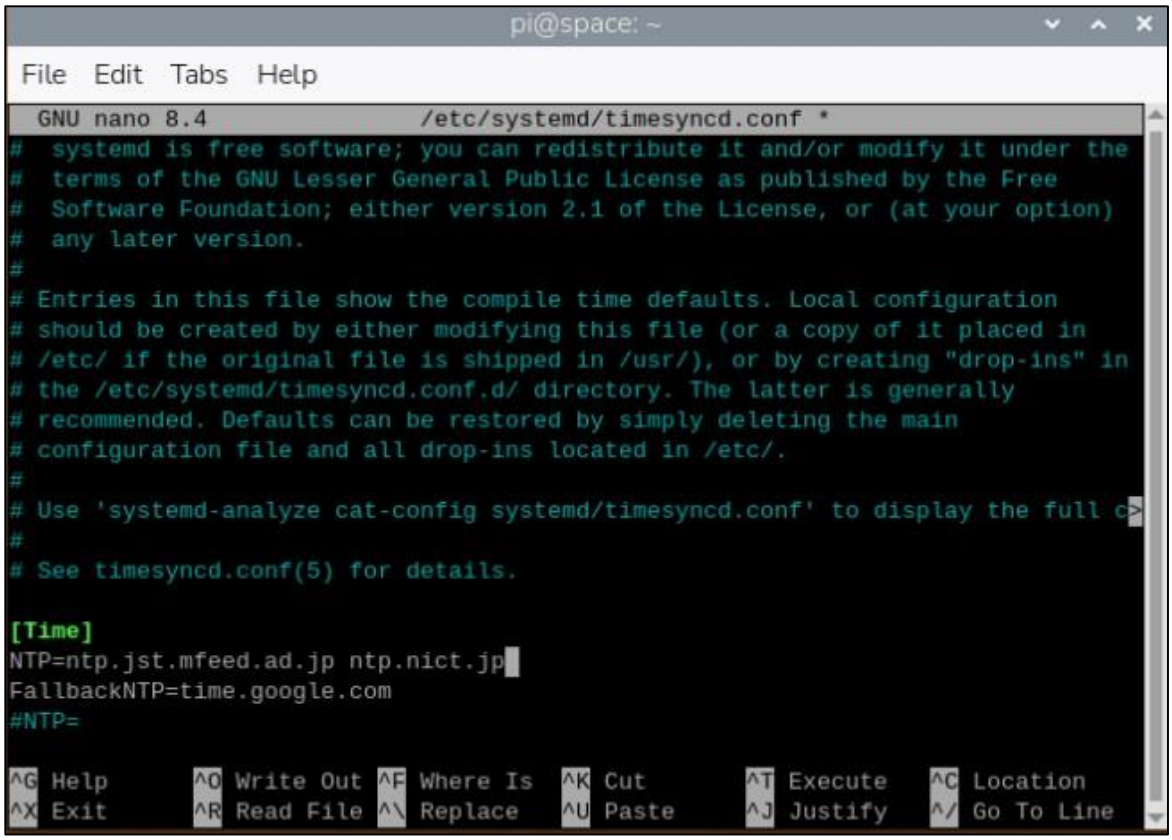
```
sudo nano /etc/systemd/timesyncd.conf
```

以下を記述して保存します（Ctrl+X → Y → Enter）：

```
[Time]
NTP=ntp.jst.mfeed.ad.jp ntp.nict.jp
FallbackNTP=time.google.com
```

設定を反映します：

```
sudo systemctl restart systemd-timesyncd
sudo systemctl status systemd-timesyncd
```



(13) OS のアップデート

VNC のターミナル（LXTerminal）または Tera Term から実行します：

```
sudo apt update
sudo apt upgrade
```

> ※ アップグレードの途中で「master_preferences (Y/I/N/O/D/Z) [default=N] ?」と表示された場合は「n」を入力してください。
>
> ※「sudo apt update でエラーが出る場合は、ラズパイのインターネット接続ができていないことになります。インターネット接続の共有の設定を再度やり直すことにより、エラーが出なくなるケースが多いです。なお、sudo apt update は、パッケージリストのアップデートだけで無害ですので、ラズパイのインターネット接続の確認にも使うことができます。」

(14) 日本語入力メソッド（fcitx5-mozc）の設定

VNC 接続時に日本語入力を使用するための設定です。

fcitx5-mozc をインストールします：

```
sudo apt install fcitx5-mozc
```

「[Y/n]」と表示されたら Enter キーを押します。

fcitx5 を有効化します：

```
im-config -n fcitx5
```

再起動して設定を反映します：

```
sudo reboot
```

再起動後、VNC 接続してタスクバーに fcitx のアイコンが表示されれば設定完了です。半角/全角キーまたは Ctrl+Space で日本語入力に切り替えられます。

> ※ LXTerminal・Chromium・Thonny で日本語入力が使用できます。

(15) alias の設定

よく使うコマンドの短縮形（alias）と PATH の設定を .bashrc に追加します。

.bashrc をエディタで開きます：

```
sudo nano .bashrc
```

ファイルの最後に以下を追記して保存します（Ctrl+X → Y → Enter）：

```
alias dir='ls -atgl | more'
alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'
alias crontab='crontab -i'
export PATH="$PATH:."
alias fastfetch='fastfetch --logo raspberrypi'
```

設定を即時反映させてください：

```
source .bashrc
```

※ rm / cp / mv / crontab に -i オプションを付けることで、上書き・削除前に確認を求めるようになり、誤操作を防ぐことができます。

```
pi@space: ~
ファイル(F)  編集(E)  タブ(T)  ヘルプ(H)
  GNU nano 8.4                          .bashrc
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi
alias dir='ls -atgl | more'
alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'
alias crontab='crontab -i'
export PATH="$PATH:."
alias fastfetch='fastfetch --logo raspberrypi'

^G ヘルプ    ^O 書き込み   ^F 検索     ^K 切り取り   ^T コマンドを ^C 位置
^X 終了      ^R 読み込み   ^\ 置換     ^U 貼り付け   ^J 均等割付   ^/ 行を指定
```

(16) OTG 接続の状態確認

OTG 接続が ON になっていることを確認します：

```
sudo rpi-usb-gadget status
```

以下のように表示されれば正常です：

```
USB Gadget mode is on
:: NetworkManager:
 iface: usb0
 link: connected
 IPv4: 192.168.137.xxx
 profile: USB Gadget (shared)
```

IPv4 のアドレスが 192.168.137.xxx になっていることを確認してください。この IP アドレスで SSH / VNC 接続できます。

(17) OTG 接続の ON/OFF 方法

通常は操作不要ですが、必要な場合は以下のコマンドを使用します。

OFF にする場合（OTG 接続ができなくなるので注意）

```
sudo rpi-usb-gadget off
sudo reboot
```

ON にする場合

実行前に必ず config.txt のバックアップを取ってください：

```
sudo cp /boot/firmware/config.txt /boot/firmware/config.txt.bak
sudo rpi-usb-gadget on
sudo reboot
```

【重要】sudo rpi-usb-gadget on 実行後の確認

`sudo rpi-usb-gadget on` を実行すると `/boot/firmware/config.txt` が書き換えられる場合があります。実行後は必ず以下のコマンドで `config.txt` の内容を確認してください：

```
cat /boot/firmware/config.txt
```

以下の行が含まれているか確認してください：

```
dtoverlay=vc4-kms-v3d
```

> ※ この行が消えていた場合、VNC が起動できなくなります（灰色画面）。その場合は config.txt に以下の内容が含まれるよう修正して再起動してください：

```
# Enable DRM VC4 V3D driver
dtoverlay=vc4-kms-v3d
max_framebuffers=2
```

(18) VNC 解像度の設定（1600×900）

Trixie 環境では labwc（Wayland コンポジター）が使用されます。labwc の autostart ファイルに wlr-randr コマンドを追加することで、起動時に自動的に解像度が設定されます。Pi5 / Pi4 / Zero2W 共通の手順です。

「VNC 解像度の変更は、ラズパイ Menu の設定の Control Centre の画面で可能ですが、設定のサポートがされていない 1600×900 の解像度にする場合は、下記の設定を行ってください。」を入

Step 1：labwc autostart ファイルを作成・編集する

```
nano ~/.config/labwc/autostart
```

以下を記述して保存してください。（Ctrl+X → Y → Enter）：

```
/usr/bin/wlr-randr --output NOOP-1 --custom-mode 1600x900 &
```

【ポイント】 末尾の「&」はバックグラウンド実行のために必要です。

【補足】HDMI 接続時の出力名について

HDMI ケーブルを接続せずに起動した場合、出力名は NOOP-1 になります。HDMI ケーブルを接続して起動した場合は出力名が変わります。

状況	出力名
HDMI なし（OTG 接続）	`NOOP-1`
HDMI 接続（HDMI0）	`HDMI-A-1`
HDMI 接続（HDMI1）Pi4 / Pi5 のみ	`HDMI-A-2`

出力名を確認するには `wlr-randr` を実行してください。出力名が異なる場合は autostart の `NOOP-1` の部分を該当の出力名に書き換えてください。

Step 2：再起動して確認する

```
sudo reboot
```

再起動後に VNC 接続して解像度が 1600×900 になっていれば設定完了です。

Step 3：SSH で解像度を確認する

```
wlr-randr
```

以下のように表示されれば成功です：

```
NOOP-1 "Headless output 2"
  Enabled: yes
  Modes:
    1600x900 px (current)
```

(19) OS アップデート後の解像度リセット対策

通常は apt upgrade 後も解像度は維持されますが、何らかの理由でリセットされる場合があります。

手順（18）で設定した ~/.config/labwc/autostart はユーザー設定ファイルのため、apt upgrade でパッケージが更新されても上書きされません。そのため OS アップデート後も解像度設定は自動的に維持されます。Pi5 / Pi4 / Zero2W 共通の手順です。

確認方法

apt upgrade 後は念のため以下で解像度を確認してください：

```
wlr-randr
```

「1600x900 px (current)」と表示されれば問題ありません。

autostart ファイルの内容確認

```
cat ~/.config/labwc/autostart
```

以下が表示されれば正常です：

```
/usr/bin/wlr-randr --output NOOP-1 --custom-mode 1600x900 &
```

解像度がリセットされた場合

万一解像度が戻った場合は、以下を確認してください：

```
ls ~/.config/labwc/
```

autostart ファイルがない場合は手順（18）を再実行してください。

config.txt の確認

`sudo rpi-usb-gadget on` を実行した場合、`config.txt` が書き換えられることがあります。実行後は必ず以下で内容を確認し、`dtoverlay=vc4-kms-v3d` が含まれているか確認してください：

```
cat /boot/firmware/config.txt
```

> ※ dtoverlay=vc4-kms-v3d が消えていた場合は config.txt を修正して再起動が必要です。

(20) GPIO21 プルアップ設定と電源オフスクリプトの自動起動（IoT 学習 HAT 使用時）

> IoT 学習 HAT を使用して GPIO21 でラズパイをシャットダウンする機能を利用する場合の設定です。
> GPIO21 を「H」にしておかないとラズパイ自体が起動できなくなります。

> ※ shutdown-off.sh を使用しない場合は、本手順は不要です。

gpio21.sh の設定（GPIO21 をプルアップ入力に設定）

1. スクリプトを作成・編集する

```
nano /home/pi/gpio21.sh
```

以下を記述して保存します（Ctrl+X → Y → Enter）：

```
#!/bin/bash
pinctrl 21 ip pu
```

2. 実行権限を付与する

```
chmod +x /home/pi/gpio21.sh
```

3. サービスファイルを作成する

```
sudo nano /etc/systemd/system/gpio21.service
```

以下を記述して保存します：

```
[Unit]
Description=GPIO21 Input Pullup
After=multi-user.target

[Service]
Type=oneshot
User=pi
ExecStart=/home/pi/gpio21.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

4. サービスを有効化・起動する

```
sudo systemctl daemon-reload
sudo systemctl enable gpio21.service
sudo systemctl start gpio21.service
```

5. 動作確認

```
sudo systemctl status gpio21.service
```

「Active: active (exited)」と表示されれば成功です。

6. 再起動して永続化を確認する

```
sudo reboot
```

再起動後に以下で確認します：

```
gpio readall
```

GPIO21 が「IN / 1」になっていれば設定完了です。

> **※ gpio コマンドは WiringPi インストール後に使用可能です。**

shutdown-off.sh の設定（GPIO21 が L になったら電源オフ）

1. スクリプトを作成・編集する

```
nano /home/pi/shutdown-off.sh
```

以下を記述して保存します（Ctrl+X → Y → Enter）：

```
#!/bin/sh
GPIO_SW=21
PUSH_TIME=3
gpio -g mode $GPIO_SW in
gpio -g mode $GPIO_SW up
count=0
while [ $count -lt $PUSH_TIME ] ; do
  data=`gpio -g read $GPIO_SW`
  if [ "$data" -eq "0" ] ; then
    count=`expr $count + 1`
  else
    count=0
  fi
  sleep 1
done
poweroff
```

2. 実行権限を付与する

```
chmod +x /home/pi/shutdown-off.sh
```

3. サービスファイルを作成する

```
sudo nano /etc/systemd/system/shutdown-off.service
```

以下を記述して保存します：

```
[Unit]
Description=Shutdown by GPIO21
After=multi-user.target gpio21.service

[Service]
Type=simple
User=root
ExecStart=/home/pi/shutdown-off.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

【ポイント】 gpio21.service の後に起動するよう「After=multi-user.target gpio21.service」を指定しています。また poweroff コマンドの実行には root 権限が必要なため「User=root」を指定しています。

4. サービスを有効化・起動する

```
sudo systemctl daemon-reload
sudo systemctl enable shutdown-off.service
sudo systemctl start shutdown-off.service
```

5. 動作確認

```
sudo systemctl status shutdown-off.service
```

「Active: active (running)」と表示されれば成功です。

6. 再起動して永続化を確認する

```
sudo reboot
```

再起動後にタクトスイッチを 3 秒間押して電源オフすれば設定完了です。

(21) 各機種の起動ビット数と config.txt の設定について

同じ MicroSD（32bit 版 OS）を各機種に挿して起動した際に、どのビット数のカーネルが起動するかを実験で確認しました。

各機種の起動ビット数（実験結果）

fastfetch または uname -m コマンドで起動ビット数を確認できます。表示の見方は以下の通りです：

```
uname -m
表示の見方：
armv6l  → 32bit カーネル
armv7l  → 32bit カーネル
aarch64 → 64bit カーネル
```

32bit 版 OS の MicroSD を各機種に挿した場合の実験結果：

```
Zero W    → armv6l（32bit カーネル）※ ARMv6 のため 64bit 非対応
Zero 2 W  → armv7l（32bit カーネル）
Pi 4      → aarch64（64bit カーネル）※ trixie では 32bit カーネルでの起動は不可
Pi 5      → aarch64（64bit カーネル）※ 32bit カーネル非対応
```

> ※ Pi 4 は trixie の 32bit 版 OS では 32bit カーネルでの起動はできません。64bit カーネルで起動します。Pi 4 は 64bit で使用してください。

Zero 2 W を 32bit カーネルで起動するための config.txt 設定

Zero 2 W は ARMv8 コアを搭載しているため、設定なしでは 64bit カーネルで起動する場合があります。32bit カーネルで起動させるには /boot/firmware/config.txt に以下を追記します：

```
[pi02]
arm_64bit=0
```

> ※ セクション名は [pi02]（ハイフンなし）が正しい書き方です。[pi0-2] では動作しません。
>
> ※ [all] に arm_64bit=0 を書くと Pi 4 が起動しなくなるため、必ず [pi02] セクションに記述してください。
>
> ※ Pi 5 は arm_64bit=0 の設定を無視して常に 64bit カーネルで起動します。

この設定は rpi-usb-gadget on を実行した後の config.txt に追記します。正しい config.txt の記述例：

```
[all]
dtoverlay=dwc2,dr_mode=peripheral

[pi02]
arm_64bit=0
```

カーネルと OS の関係

よく使われる例えとして「カーネル＝エンジン、OS＝車全体」があります。

カーネル（核）

ハードウェアと直接会話する唯一の存在です。CPU、メモリ、GPIO、USB などをすべてカーネルが管理します。Raspberry Pi で言えば `/boot/firmware/` にある `kernel8.img`（64bit）や `kernel7l.img`（32bit）がそれです。`uname -m` で表示されるのはこのカーネルのビット数です。

ユーザーランド（OS の残り部分）

カーネルの上で動くすべてのもの、つまり `ls`、`apt`、Python、LXDE（GUI）、あなたが書いたスクリプトなど全部です。カーネルに「ファイルを読みたい」「画面に表示したい」とお願いして動きます。

なぜ「32bit 版 OS で 64bit カーネル起動」が起きるのか

Raspberry Pi 4 でまさにこれが起きています。

```
32bit 版 OS イメージ
├── kernel（→ Pi4 は Trixie で 64bit カーネルが選ばれる）
└── ユーザーランド（32bit 用バイナリ群）
    ├── /bin/ls    （32bit）
    └── /usr/bin/python3    （32bit）
```

64bit カーネルは 32bit のユーザーランドのプログラムも実行できます（互換性あり）。逆はできません。なので `uname -m` が `aarch64`（64bit）でも、アプリは 32bit で動いているという少し不思議な状態になります。

Raspberry Pi 各機種での具体的な動き

機種	CPU	32bit 版 OS を挿したとき
Zero W	ARMv6	64bit 非対応のため 32bit カーネルで起動
Zero 2W	ARMv8	設定次第で 32bit/64bit どちらにもなる
Pi 4	ARMv8	Trixie では強制的に 64bit カーネル
Pi 5	ARMv8	常に 64bit カーネル（32bit 設定を無視）

Zero 2W だけが ARMv8 なのに 32bit カーネルで起動できる（`[pi02] arm_64bit=0` の設定で）のは、ファームウェアが明示的に指示されたときだけ 32bit カーネルを選ぶためです。

まとめ

・カーネルのビット数 = `uname -m` で見える、ハードウェア制御の土台

・ユーザーランドのビット数 = インストールした OS イメージが 32bit 版か 64bit 版か

・両者は独立して選べる（ただしカーネルが 64bit ならユーザーランドは 32bit/64bit どちらでも OK、逆は不可）

> 手順書の「32bit 版 OS でも 64bit カーネルで起動できる」という記述はまさにこの仕組みを指しています！

カスタマイズ（任意）

以下は必須ではありませんが、便利なツールのインストール手順です。

fastfetch のインストール

システム情報を見やすく表示するツールです。

```
sudo apt install fastfetch
```

実行方法：

```
fastfetch
```

gparted のインストール

GUI でディスクのパーティション管理ができるツールです。

```
sudo apt install gparted
```

実行方法（VNC 接続が必要）：

```
sudo gparted
```

WiringPi のインストール

GPIO 制御ライブラリです。Trixie では apt では入らないため、GitHub からソースをビルドしてインストールします。

必要なパッケージをインストールします：

```
sudo apt install git
```

GitHub からソースを取得します：

```
git clone https://github.com/WiringPi/WiringPi.git
```

ディレクトリに移動します：

```
cd WiringPi
```

ビルドして deb パッケージを作成します：

```
./build debian
```

deb パッケージをインストールします：

```
sudo dpkg -i debian-template/wiringpi*.deb
```

インストールを確認します：

```
gpio -v
gpio readall
```

> ※ WiringPi は Pi 5 でも動作しますが、GCLK 機能は未対応です。

トラブルシューティング

症状	原因	対処法
デバイスが認識されない	RNDIS ドライバ未インストール	手順（1）のドライバをインストール
SSH 接続できない	起動未完了 / IP 不明	5 分待つ、ホスト名で再試行
VNC 接続拒否	wayvnc 未起動	sudo systemctl status wayvnc で確認
VNC が灰色画面になる	config.txt の設定不備	dtoverlay=vc4-kms-v3d が含まれているか確認
解像度がリセットされる	autostart ファイルがない	autostart がない場合は手順（18）を再実行
apt upgrade 後に解像度リセット	autostart ファイルがない	手順（19）を確認し、必要に応じて手順（18）を再実行
OTG 接続が切れる	USB 電源不足（Pi5）	データ転送対応ケーブルを使用
GPIO21 が L のまま起動した場合	shutdown-off.sh が起動し 3 秒後に poweroff される	Image を作り直して手順（3）～（19）を再実行する

便利な確認コマンド

説明	コマンド
OTG 接続の状態確認	`sudo rpi-usb-gadget status`
OTG IP アドレス確認	`ip a show usb0`
VNC サーバー状態確認	`sudo systemctl status wayvnc`
VNC ログ確認	`sudo journalctl -u wayvnc -n 50`
現在の VNC 解像度確認 出力先名確認	`wlr-randr`
OS バージョン確認	`cat /etc/os-release`
GPIO 状態確認	`gpio readall`
gpio21 サービス確認	`sudo systemctl status gpio21.service`
shutdown-off サービス確認	`sudo systemctl status shutdown-off.service`
起動ビット数確認	`uname -m`

© 2026 @M164

この文書は、クリエイティブ・コモンズ 表示 4.0 国際 ライセンス（CC BY 4.0）のもとで提供されています。
https://creativecommons.org/licenses/by/4.0/deed.ja